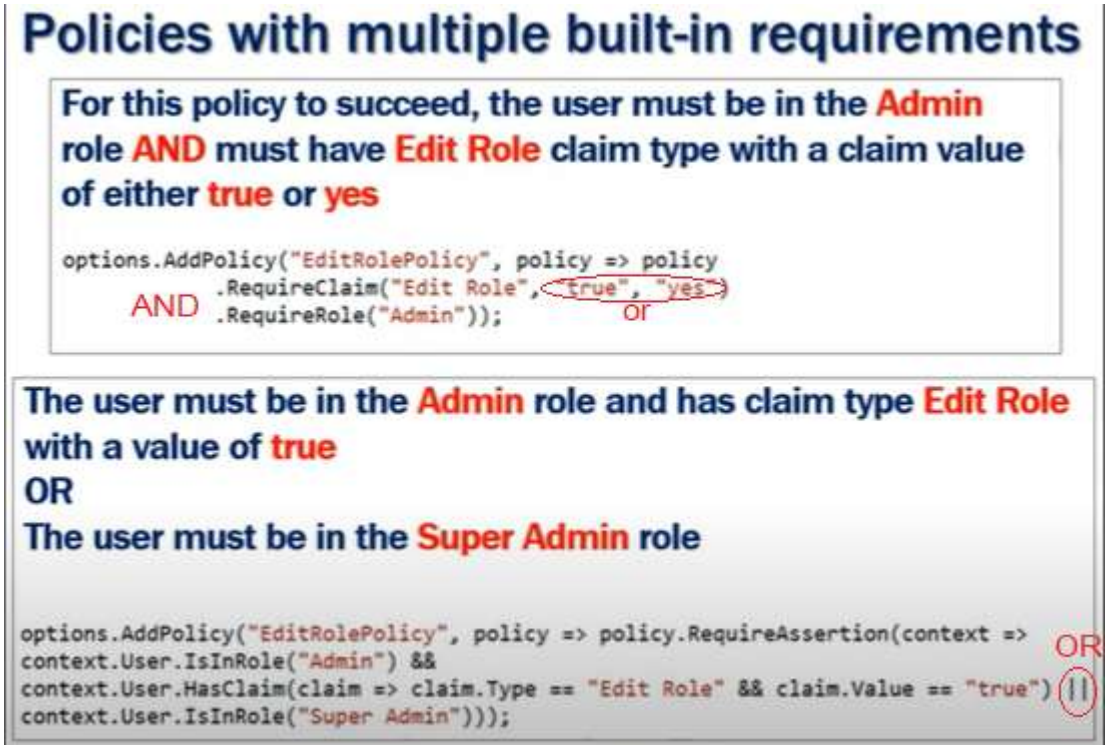
100 - Custom authorization requirements and handlers

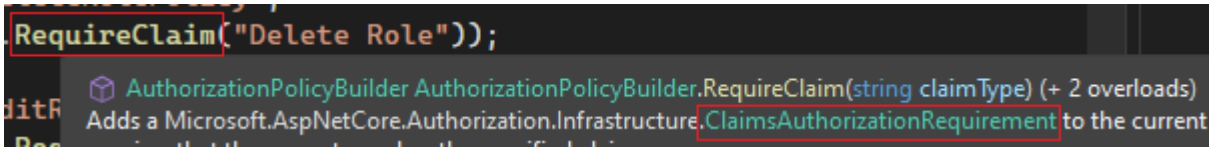
Simple requirements:

```
options.AddPolicy("EditRolePolicy",
    policy => policy.RequireClaim("Edit Role"));

options.AddPolicy("EditRolePolicy",
    policy => policy.RequireClaim("Edit Role", "true"));

options.AddPolicy("EditRolePolicy",
    policy => policy.RequireClaim("Edit Role", "true", "yes"));
```

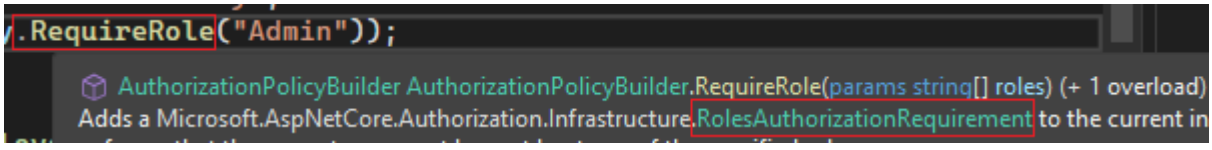
Multiple requirements:



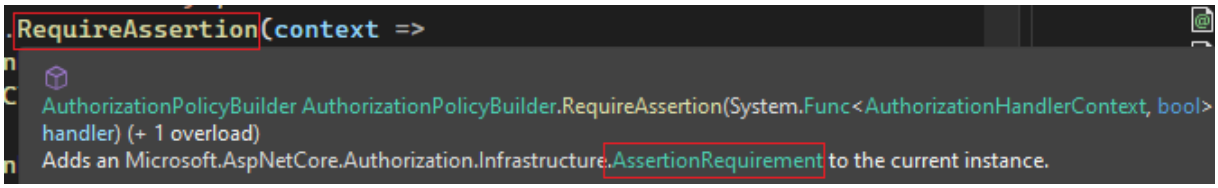
От Intellysense се вижда, че Build-in Requirements са съответно:
.RequireClaim - .ClaimsAuthorizationRequirement



.RequireRole - .RolesAuthorizationRequirement



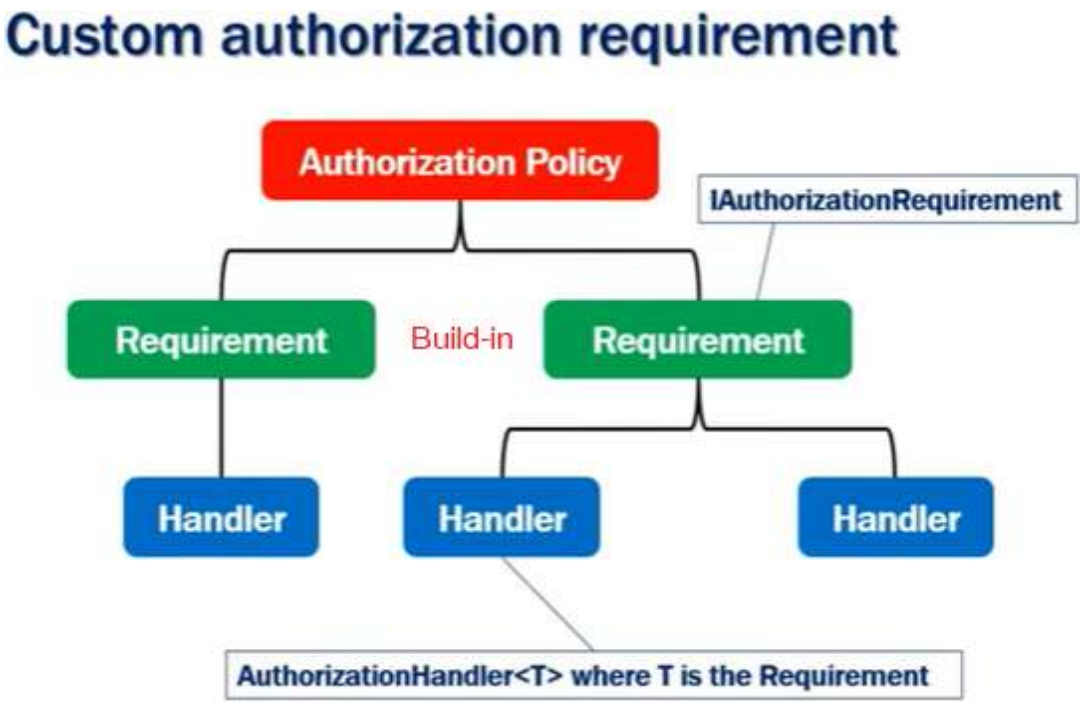
.RequireAsserion - .AssertionRequirement



Built-in asp.net core authorization requirements

- RequireClaim() method adds ClaimsAuthorizationRequirement.
- RequireRole() method adds RolesAuthorizationRequirement.
- RequireAssertion() method adds AssertionRequirement.

За прости приложения, тези Requirements ще свършат работа, но за по-сложни, трябва да въведем персонализирани:



An authorization policy has one or more requirements. Each authorization requirement has one or more handlers.

All the built-in authorization requirements implement IAuthorizationRequirement interface. So to create a custom authorization requirement, we need to implement IAuthorizationRequirement interface. This is an empty marker interface, which means there is nothing in this interface that our custom requirement class must implement.

It is in the authorization handler that we write our logic to allow or deny access to a resource like a controller action for example. An authorization handler implements AuthorizationHandler<T> where T is the type of requirement.

For example, let's say, an admin user can assign or remove roles of other admin users but not his own roles. To achieve this, we need to know the logged-in UserID and the UserId of

the Admin being edited. If they are the same we do not want to allow access. The admin UserID being edited is passed in the URL as a query string parameter. From the authorization handler we will have access to the route data and URL query string parameters. Dependency injection is also supported. This means, we can even inject and use other services if required.